# The Manconomy — Solution Architecture Document

> *“Trade like a man. Live like a neighbor.”*

**Version:** 1.0
**Date:** April 11, 2026
**Status:** Pre-Build — Ready for Developer Handoff
**Author:** Founder

---

## Table of Contents


1. System Overview
2. Component Architecture
3. Data Model
4. API Design
5. AI Pipeline
6. Credit Ledger Architecture
7. Security Model
8. Infrastructure
9. Key Technical Decisions
10. Open Questions


---

## 1. System Overview

### 1.1 What This System Does

The Manconomy is a hyperlocal, credit-based barter marketplace. Users photograph items they want to trade, AI generates listings with suggested credit valuations, and neighbors within a configurable radius can trade. Credits are a closed-loop internal currency — never convertible to cash — that solve the “double coincidence of wants” problem inherent in direct barter.

### 1.2 System Context Diagram

```
┌──────────────────────────────────────────────────────────────────────┐
│                          EXTERNAL USERS                              │
│                                                                      │
│   iOS User ────┐                            Android User ────┐        │
│                │                                            │        │
└────────────────┼────────────────────────────────────────────┼────────┘
                 │                                            │
                 ▼                                            ▼
┌──────────────────────────────────────────────────────────────────────┐
│                      MOBILE APPLICATION                              │
│                React Native + Expo (single codebase)                  │
│                                                                      │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────────────┐    │
│  │  Expo    │  │ Zustand  │  │  Expo    │  │ react-native-    │    │
│  │  Router  │  │  State   │  │  Push    │  │    maps          │    │
│  └──────────┘  └──────────┘  └──────────┘  └──────────────────┘    │
└──────────────────────────────────────────────────────────────────────┘
                 │
                 │ HTTPS / WebSocket
                 ▼
┌──────────────────────────────────────────────────────────────────────┐
│                      SUPABASE (Backend-as-a-Service)                 │
│                                                                      │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────────┐   │
│  │  Supabase    │  │  Supabase    │  │   Supabase Realtime      │   │
│  │  Auth        │  │  PostGres    │  │   (WebSocket pub/sub)    │   │
│  │  (JWT/OAuth) │  │  + PostGIS   │  │                          │   │
│  └──────────────┘  └──────────────┘  └──────────────────────────┘   │
│                                                                      │
│  ┌──────────────┐  ┌──────────────────────────────────────────┐     │
│  │  Supabase    │  │         Supabase Edge Functions          │     │
│  │  Storage     │  │         (Deno / TypeScript)              │     │
│  │  (photos)    │  │  ai-listing / credit-transfer / rank-update  │  │
│  └──────────────┘  └──────────────────────────────────────────┘     │
└──────────────────────────────────────────────────────────────────────┘
                 │
                 │ HTTPS API calls (server-side only)
                 ▼
┌──────────────────────────────────────────────────────────────────────┐
│                      EXTERNAL SERVICES                               |
│                                                                      │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────┐       │
│  │  OpenAI      │  │  eBay Browse │  │  Google Cloud Vision │       │
│  │  GPT-4o Vision│  │  API         │  │  SafeSearch          │       │
```

```
│  │  GPT-4o-mini  │  │  (free)       │  │  (1K/mo free)          │  │
│  │  Moderation   │  │               │  │                        │  │
│  └───────────────┘  └───────────────┘  └────────────────────────┘  │
│                                                                    │
│  ┌───────────────┐  ┌───────────────┐  ┌────────────────────────┐  │
│  │  Stripe       │  │  PostHog      │  │  Sentry                │  │
│  │  (payments)   │  │  (analytics)  │  │  (error tracking)      │  │
│  └───────────────┘  └───────────────┘  └────────────────────────┘  │
└────────────────────────────────────────────────────────────────────┘
```

### 1.3 Core Architectural Principles

1. **Server-authoritative on money.** All credit mutations happen in server-side RPCs or Edge Functions. The client is never trusted to calculate or report balances.
2. **Credits are a ledger, not a column.** User credit balance is always calculated from the `credits_ledger` table. There is no `balance` column on the users table.
3. **Immutable trade records.** Completed trades are never deleted or modified — status transitions only. This is the audit trail.
4. **RLS everywhere.** Supabase Row Level Security policies enforce that users can only read/write data they own or have a relationship to. Security is in the database, not just the API.
5. **AI pipeline is server-side.** OpenAI API keys are never in the client bundle. All AI calls originate from Edge Functions.
6. **Offline-tolerant reads, online-required writes.** The feed can be cached for offline browsing. Any write (trade, message, listing) requires network connectivity.

---

## 2. Component Architecture

### 2.1 Mobile App (React Native + Expo)

#### File Structure (Expo Router — file-based routing)

```
app/
  (auth)/
    login.tsx           — Email/phone login
    register.tsx        — New user signup
    invite.tsx          — Invite code validation
    onboarding/
      index.tsx         — Screen 1: value prop
      neighborhood.tsx  — Screen 2: set neighborhood
      credits.tsx       — Screen 3: seed credits explained
      first-listing.tsx — Screen 4: prompt to list
  (tabs)/
```

```
    index.tsx           — The Market (main feed)
    search.tsx          — Search + filters
    add.tsx             — New listing (camera → AI → review)
    messages.tsx        — Trade conversations
    profile.tsx         — Own profile / The Sash
  listing/
    [id].tsx            — Listing detail page
    [id]/edit.tsx       — Edit listing
    [id]/pitches.tsx    — The Pitch board for a listing
  trade/
    [id].tsx            — Trade detail / coordination
    [id]/confirm.tsx    — Dual confirmation screen
  user/
    [id].tsx            — Other user's profile
  leaderboard.tsx       — Neighborhood leaderboards
  climb.tsx             — Active Climbs feed
  settings.tsx          — Account settings
  wallet.tsx            — Credit wallet + transaction history

components/
  listing/
    ListingCard.tsx     — Feed tile
    ListingDetail.tsx   — Full listing view
    AISealBadge.tsx     — Trust badge
    FoodUrgencyBadge.tsx — Perishable countdown
  trade/
    TradeRequest.tsx    — Trade request form
    MeetupCoord.tsx     — Location/time picker
    DualConfirm.tsx     — Both-party confirmation
  profile/
    TheSash.tsx         — Patches display
    MeritRankBadge.tsx  — Rank badge
    TradeHistory.tsx    — Past trades list
  credits/
    WalletBalance.tsx   — Balance display (header)
    CreditTransaction.tsx — Ledger entry display
  common/
    PatchUnlock.tsx     — Achievement toast
    RankUp.tsx          — Rank promotion modal

lib/
  supabase.ts           — Supabase client init
  api.ts                — Typed API wrapper functions
  store.ts              — Zustand global state
  notifications.ts      — Expo push notification setup
  geo.ts                — Distance/geofence utilities
  credits.ts            — Credit display/formatting helpers
```

```
types/
  index.ts              — All TypeScript types (User, Listing, Trade,
etc.)
```

**State Management (Zustand)**

```typescript
// store.ts — global slices
interface AppState {
  // Auth
  user: User | null;
  session: Session | null;

  // Feed
  listings: Listing[];
  feedRadius: number;       // miles: 0.5 | 1 | 2 | 3
  feedFilters: FeedFilters;

  // Wallet (optimistic balance display)
  creditBalance: number;    // Always re-fetched from server on mount

  // Active trade
  activeTrade: Trade | null;

  // Actions
  setUser: (user: User) => void;
  setListings: (listings: Listing[]) => void;
  setCreditBalance: (n: number) => void;
  setActiveTrade: (trade: Trade | null) => void;
}
```

> ***Rule:*** `creditBalance` *in Zustand is for display only. Any action that depends on a user having enough credits must call the server-side RPC, which recalculates from the ledger. Never gate on client-side balance.*

---

## 2.2 Backend (Supabase)

**Supabase Edge Functions (Deno/TypeScript)**

All business logic that involves credit mutation, AI calls, or multi-table writes runs in Edge Functions. These are invoked from the mobile client via the Supabase client SDK.

| Function | Trigger | Description |
|---|---|---|
| `ai-listing` | Client POST | Receives photo URL + user ID. Calls GPT-4o Vision + GPT-4o-mini + eBay API. Returns structured listing data. |
| `create-listing` | Client POST | Creates listing record, awards first-listing bonus if applicable. |
| `request-trade` | Client POST | Creates trade record, validates buyer credit balance, holds escrow. |
| `accept-trade` | Client POST | Transitions trade to accepted. Sends push to buyer. |
| `confirm-trade` | Client POST | Records one party's confirmation. If both confirmed: executes credit transfer, awards patches, updates rank. |
| `auto-confirm-trade` | Cron (every 6hr) | Auto-confirms trades where one party confirmed >48 hours ago and the other hasn't responded. |
| `cancel-trade` | Client POST | Cancels trade, releases escrow credits back to buyer. |
| `purchase-credits` | Webhook (Stripe) | Receives Stripe payment confirmation. Mints credits to buyer wallet via ledger entry. |
| `boost-listing` | Client POST | Deducts 15 credits from user, sets `boost_until` on listing. |
| `update-rank` | Called internally | Recalculates merit rank for a user after any qualifying event. |
| `award-patch` | Called internally | Idempotently awards a patch if not already earned. |
| `moderation-scan` | Called on listing create | Runs text through OpenAI Moderation API + image through Google SafeSearch. Returns pass/flag. |
| `decay-inactive-credits` | Cron (monthly) | Applies 10% balance decay to accounts inactive >90 days. Sends push notification 7 days before. |

| Function | Trigger | Description |
|---|---|---|
| `generate-1099b` | Cron (Jan 1) | Exports annual trade records for users who crossed $600 FMV threshold. |

---

## 3. Data Model

### 3.1 Full Schema

All tables in PostgreSQL via Supabase. PostGIS extension required for geographic queries.

---

**`neighborhoods`**

```
CREATE TABLE neighborhoods (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name          TEXT NOT NULL,
  city          TEXT NOT NULL,
  state         TEXT NOT NULL,
  center_lat    DECIMAL(10, 7) NOT NULL,
  center_lng    DECIMAL(10, 7) NOT NULL,
  geofence      GEOGRAPHY(POLYGON, 4326),  -- PostGIS polygon
  member_count  INTEGER DEFAULT 0,
  status        TEXT DEFAULT 'forming'     -- forming | active | open
                CHECK (status IN ('forming', 'active', 'open')),
  created_at    TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX ON neighborhoods USING GIST (geofence);
```

---

**`users`**

```
CREATE TABLE users (
  id                UUID PRIMARY KEY REFERENCES auth.users ON DELETE
        CASCADE,
  display_name      TEXT NOT NULL,
  avatar_url        TEXT,
  neighborhood_id   UUID REFERENCES neighborhoods(id),
  subscription_tier TEXT DEFAULT 'free'
```

```
                    CHECK (subscription_tier IN ('free', 'pro',
        'legend')),
  merit_rank        INTEGER DEFAULT 0 CHECK (merit_rank BETWEEN 0 AND 8),
  trade_count       INTEGER DEFAULT 0,
  avg_rating        DECIMAL(3, 2) DEFAULT 0.0,
  is_verified       BOOLEAN DEFAULT false,        -- phone verified
  is_charter_member BOOLEAN DEFAULT false,
  member_since      TIMESTAMPTZ DEFAULT now(),
  last_active_at    TIMESTAMPTZ DEFAULT now(),
  -- Tax compliance
  tax_id_collected  BOOLEAN DEFAULT false,
        -- SSN/EIN collected for 1099-B
  ytd_trade_value   INTEGER DEFAULT 0,            -- implied cents for
        1099-B tracking
  -- Feature flags (subscription limits)
  max_active_listings INTEGER DEFAULT 5,
  max_photos_per_listing INTEGER DEFAULT 3,
  ai_listings_this_month INTEGER DEFAULT 0,
  ai_listing_limit  INTEGER DEFAULT 10,
  created_at        TIMESTAMPTZ DEFAULT now(),
  updated_at        TIMESTAMPTZ DEFAULT now()
);

-- Computed credit balance: use credits_ledger, not a column here.
-- Balance function:
CREATE OR REPLACE FUNCTION get_credit_balance(p_user_id UUID)
RETURNS INTEGER AS $$
  SELECT COALESCE(SUM(amount), 0)
  FROM credits_ledger
  WHERE to_user = p_user_id
  UNION ALL
  SELECT -COALESCE(SUM(amount), 0)
  FROM credits_ledger
  WHERE from_user = p_user_id;
$$ LANGUAGE SQL STABLE SECURITY DEFINER;
```

---

**listings**

```
CREATE TABLE listings (
  id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id             UUID NOT NULL REFERENCES users(id) ON DELETE
        CASCADE,
  neighborhood_id     UUID NOT NULL REFERENCES neighborhoods(id),
  title               TEXT NOT NULL,
  description         TEXT,
  category            TEXT NOT NULL
```

```sql
                        CHECK (category IN (
                          'tools', 'electronics', 'outdoor_camping',
                          'sports_equipment', 'home_garage',
        'vehicles_parts',
                          'gaming', 'food_provision', 'other'
                        )),
  condition             TEXT CHECK (condition IN
        ('like_new','good','fair','for_parts')),
  credit_price          INTEGER NOT NULL CHECK (credit_price >= 0),
  photos                TEXT[] NOT NULL,           -- array of storage URLs
  -- AI fields
  ai_seal               BOOLEAN DEFAULT false,
  ai_suggested_price    INTEGER,
  ai_confidence         DECIMAL(4,3),              -- 0.000 to 1.000
  crowd_valuation       INTEGER,                   -- running avg from
        evaluations
  crowd_rating_count    INTEGER DEFAULT 0,
  -- Status
  status                TEXT DEFAULT 'active'
                        CHECK (status IN
        ('active','reserved','traded','expired','removed')),
  boost_until           TIMESTAMPTZ,
  expires_at            TIMESTAMPTZ DEFAULT (now() + INTERVAL '30 days'),
  -- Pitch mode
  pitch_mode            BOOLEAN DEFAULT false,
  pitch_window_hours    INTEGER DEFAULT 48,
  pitch_closes_at       TIMESTAMPTZ,
  -- Food fields (populated only for food/perishable listings)
  is_perishable         BOOLEAN DEFAULT false,
  food_category         TEXT CHECK (food_category IN (
                          'produce','baked_goods','preserved_smoked',

        'homebrewed','packaged','cooking_gear','tradework'
                        )),
  pickup_by_date        DATE,
  food_safety_ack       BOOLEAN DEFAULT false,     -- user acknowledged
        disclaimer
  -- Climb mode
  climb_chain_id        UUID REFERENCES climb_chains(id),
  climb_step_number     INTEGER,
  -- Timestamps
  created_at            TIMESTAMPTZ DEFAULT now(),
  updated_at            TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX ON listings (neighborhood_id, status, created_at DESC);
CREATE INDEX ON listings (neighborhood_id, boost_until DESC NULLS LAST);
CREATE INDEX ON listings (user_id);
CREATE INDEX ON listings (expires_at) WHERE status = 'active';
```

`trades`

```sql
CREATE TABLE trades (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  listing_id        UUID NOT NULL REFERENCES listings(id),
  buyer_id          UUID NOT NULL REFERENCES users(id),
  seller_id         UUID NOT NULL REFERENCES users(id),
  offer_type        TEXT NOT NULL CHECK (offer_type IN
        ('credit','cash','swap')),
  credit_amount     INTEGER,
  cash_amount       INTEGER,                -- cents (e.g. 500 = $5.00)
  platform_fee_pct  INTEGER,                -- e.g. 12 for 12%
  platform_fee_amt  INTEGER,                -- cents
  -- Trade lifecycle
  status            TEXT DEFAULT 'requested'
                    CHECK (status IN (
                      'requested','accepted','countered',
                      'completed','cancelled','disputed'
                    )),
  -- Meetup coordination
  meetup_location   TEXT,
  meetup_lat        DECIMAL(10,7),
  meetup_lng        DECIMAL(10,7),
  meetup_time       TIMESTAMPTZ,
  -- Dual confirmation
  buyer_confirmed   BOOLEAN DEFAULT false,
  seller_confirmed  BOOLEAN DEFAULT false,
  auto_confirm_at   TIMESTAMPTZ,            -- set to created_at + 48hr on
        acceptance
  -- Food add-on (JSONB for flexibility)
  food_addons       JSONB,
  -- e.g. [{"description":"6-pack
        IPA","type":"homebrewed","from":"buyer"}]
  -- Timestamps
  created_at        TIMESTAMPTZ DEFAULT now(),
  updated_at        TIMESTAMPTZ DEFAULT now(),
  completed_at      TIMESTAMPTZ,
  cancelled_at      TIMESTAMPTZ,

  CONSTRAINT no_self_trade CHECK (buyer_id != seller_id)
);

CREATE INDEX ON trades (buyer_id, status);
CREATE INDEX ON trades (seller_id, status);
```

```sql
CREATE INDEX ON trades (listing_id);
CREATE INDEX ON trades (auto_confirm_at) WHERE status = 'accepted';
```

---

`credits_ledger`

```sql
CREATE TABLE credits_ledger (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  from_user         UUID REFERENCES users(id),    -- NULL = system
        (minting)
  to_user           UUID NOT NULL REFERENCES users(id),
  amount            INTEGER NOT NULL CHECK (amount > 0),
  type              TEXT NOT NULL CHECK (type IN (
                      'trade_transfer',           -- buyer → seller on
        completion
                      'escrow_hold',              -- buyer → escrow on
        trade accept
                      'escrow_release',           -- escrow → seller on
        completion
                      'escrow_return',            -- escrow → buyer on
        cancellation
                      'signup_bonus',             -- system → user
                      'first_listing_bonus',      -- system → user
                      'evaluation_reward',        -- system → user (Tinder
        for Blenders)
                      'story_reward',             -- system → user (Item
        Stories)
                      'referral_bonus',           -- system → user
                      'rating_reward',            -- system → user (5-star
        received)
                      'food_addon_reward',        -- system → user
                      'garden_listing_bonus',     -- system → user
                      'food_trade_bonus',         -- system → user
                      'insider_review_reward',    -- system → linked
        account
                      'subscription_bonus',       -- system → user
        (monthly)
                      'iap_purchase',             -- system → user (Stripe
        IAP)
                      'boost_fee',                -- user → system
        (destroyed)
                      'relist_fee',               -- user → system
        (destroyed)
                      'decay'                     -- balance reduction
        (inactive)
                    )),
  reference_id      UUID,                         -- trade_id or other
        reference
  idempotency_key   UUID NOT NULL UNIQUE,         -- prevent double-spend
  created_at        TIMESTAMPTZ DEFAULT now()
);
```

```sql
-- Balance index — used by get_credit_balance() frequently
CREATE INDEX ON credits_ledger (to_user);
CREATE INDEX ON credits_ledger (from_user);
CREATE INDEX ON credits_ledger (created_at DESC);
```

> ***Architecture note:*** *The* `escrow_hold` *type represents credits taken from a buyer when they initiate a trade. The buyer's spendable balance is* `balance - SUM(escrow_holds_in_active_trades)`*. The* `get_credit_balance()` *function should subtract escrow holds. Release happens on completion (→ seller) or cancellation (→ buyer back).*

---

### `ratings`

```sql
CREATE TABLE ratings (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  trade_id    UUID NOT NULL REFERENCES trades(id),
  rater_id    UUID NOT NULL REFERENCES users(id),
  rated_id    UUID NOT NULL REFERENCES users(id),
  stars       INTEGER NOT NULL CHECK (stars BETWEEN 1 AND 5),
  comment     TEXT,
  created_at  TIMESTAMPTZ DEFAULT now(),

  UNIQUE (trade_id, rater_id)
);
```

---

### `evaluations` (Tinder for Blenders)

```sql
CREATE TABLE evaluations (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  listing_id  UUID NOT NULL REFERENCES listings(id) ON DELETE CASCADE,
  user_id     UUID NOT NULL REFERENCES users(id),
  verdict     TEXT NOT NULL CHECK (verdict IN
        ('fair','too_high','too_low','steal','skip')),
  comment     TEXT,
  created_at  TIMESTAMPTZ DEFAULT now(),

  UNIQUE (listing_id, user_id)
);
```

```
CREATE INDEX ON evaluations (listing_id);
CREATE INDEX ON evaluations (user_id, created_at DESC);  -- for daily cap
        enforcement
```

---

`insider_accounts`

```
CREATE TABLE insider_accounts (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  insider_user_id UUID NOT NULL REFERENCES users(id),   -- the partner
  linked_user_id  UUID NOT NULL REFERENCES users(id),   -- the trader
  status          TEXT DEFAULT 'active' CHECK (status IN
        ('active','unlinked')),
  created_at      TIMESTAMPTZ DEFAULT now(),

  UNIQUE (insider_user_id, linked_user_id),
  CONSTRAINT no_self_link CHECK (insider_user_id != linked_user_id)
);
```

---

`insider_reviews`

```
CREATE TABLE insider_reviews (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  insider_id        UUID NOT NULL REFERENCES users(id),
  listing_id        UUID NOT NULL REFERENCES listings(id),
  condition_accurate BOOLEAN,
  price_fair        TEXT CHECK (price_fair IN
        ('too_low','fair','too_high')),
  notes             TEXT,
  credits_awarded   INTEGER DEFAULT 3,
  created_at        TIMESTAMPTZ DEFAULT now(),

  UNIQUE (insider_id, listing_id)
);
```

---

`messages`

```
CREATE TABLE messages (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  trade_id    UUID NOT NULL REFERENCES trades(id) ON DELETE CASCADE,
  sender_id   UUID NOT NULL REFERENCES users(id),
```

```
  body        TEXT NOT NULL,
  created_at  TIMESTAMPTZ DEFAULT now(),
  read_at     TIMESTAMPTZ
);

CREATE INDEX ON messages (trade_id, created_at ASC);
```

---

**patches**

```
CREATE TABLE patches (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id       UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  patch_key     TEXT NOT NULL CHECK (patch_key IN (
                  'first_fire',
                  'the_handshake',
                  'provisioner',
                  'the_gardener',
                  'brewmaster',
                  'storyteller',
                  'the_connector',
                  'the_climb',
                  'quartermasters_choice',
                  'the_nudge',
                  'clean_sweep',
                  'founding_member',
                  'the_grill',
                  'eagle'
                )),
  earned_at     TIMESTAMPTZ DEFAULT now(),
  display_order INTEGER,

  UNIQUE (user_id, patch_key)
);
```

---

**merit_rank_thresholds**

```
-- Static reference table. Seeded at migration time.
CREATE TABLE merit_rank_thresholds (
  rank_level         INTEGER PRIMARY KEY CHECK (rank_level BETWEEN 0 AND
        8),
  title              TEXT NOT NULL,
  min_trades         INTEGER DEFAULT 0,
```

```sql
  min_avg_rating     DECIMAL(3,2) DEFAULT 0.0,
  min_tenure_days    INTEGER DEFAULT 0,
  min_referrals      INTEGER DEFAULT 0
);

INSERT INTO merit_rank_thresholds VALUES
  (0, 'Tenderfoot',   0,   0.0,  0,   0),
  (1, 'Scout',        1,   0.0,  0,   0),
  (2, 'Trader',       5,   4.0,  0,   0),
  (3, 'Dealer',       15,  4.2,  30,  0),
  (4, 'Outfitter',    30,  4.5,  60,  1),
  (5, 'Trailblazer',  60,  4.6,  120, 2),
  (6, 'Quartermaster',100, 4.7,  180, 3),
  (7, 'Captain',      200, 4.8,  270, 5),
  (8, 'Legend',       NULL,4.9,  365, 10);  -- Legend: top 1% + community
        vote
```

---

`listing_comments`

```sql
CREATE TABLE listing_comments (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  listing_id        UUID NOT NULL REFERENCES listings(id) ON DELETE
        CASCADE,
  user_id           UUID NOT NULL REFERENCES users(id),
  body              TEXT NOT NULL,
  parent_comment_id UUID REFERENCES listing_comments(id),  -- threading
  upvotes           INTEGER DEFAULT 0,
  created_at        TIMESTAMPTZ DEFAULT now(),
  updated_at        TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX ON listing_comments (listing_id, created_at ASC);
```

---

`pitches`

```sql
CREATE TABLE pitches (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  listing_id  UUID NOT NULL REFERENCES listings(id) ON DELETE CASCADE,
  buyer_id    UUID NOT NULL REFERENCES users(id),
  pitch_body  TEXT NOT NULL CHECK (char_length(pitch_body) >= 30),
  why_me      TEXT,                          -- optional second prompt
  credits_offered INTEGER NOT NULL,
```

```
  status      TEXT DEFAULT 'pending'
              CHECK (status IN ('pending','selected','rejected')),
  vote_count  INTEGER DEFAULT 0,
  created_at  TIMESTAMPTZ DEFAULT now(),

  UNIQUE (listing_id, buyer_id)             -- one pitch per buyer per
        listing
);

CREATE INDEX ON pitches (listing_id, vote_count DESC);
```

---

**item_stories**

```
CREATE TABLE item_stories (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  listing_id    UUID NOT NULL REFERENCES listings(id) ON DELETE CASCADE,
  user_id       UUID NOT NULL REFERENCES users(id),
  prompt_key    TEXT NOT NULL,              -- e.g. 'what_did_you_love'
  prompt_text   TEXT NOT NULL,             -- full prompt as shown to user
  response_text TEXT NOT NULL CHECK (char_length(response_text) >= 15),
  credits_awarded INTEGER DEFAULT 6,
  reaction_count INTEGER DEFAULT 0,
  created_at    TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX ON item_stories (listing_id);
```

---

**climb_chains**

```
CREATE TABLE climb_chains (
  id                    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  initiator_user_id     UUID NOT NULL REFERENCES users(id),
  neighborhood_id       UUID NOT NULL REFERENCES neighborhoods(id),
  declared_goal         TEXT,
  starting_item         TEXT NOT NULL,
  starting_value        INTEGER NOT NULL,
  current_value         INTEGER,
  chain_length          INTEGER DEFAULT 0,
  status                TEXT DEFAULT 'active'
                        CHECK (status IN
        ('active','completed','abandoned')),
  started_at            TIMESTAMPTZ DEFAULT now(),
```

```
  completed_at          TIMESTAMPTZ
);
```

---

`climb_steps`

```
CREATE TABLE climb_steps (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  chain_id          UUID NOT NULL REFERENCES climb_chains(id),
  step_number       INTEGER NOT NULL,
  from_user_id      UUID NOT NULL REFERENCES users(id),
  to_user_id        UUID NOT NULL REFERENCES users(id),
  trade_id          UUID REFERENCES trades(id),
  item_description  TEXT NOT NULL,
  credit_value      INTEGER NOT NULL,
  created_at        TIMESTAMPTZ DEFAULT now()
);
```

---

`leaderboard_snapshots`

```
CREATE TABLE leaderboard_snapshots (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  neighborhood_id   UUID NOT NULL REFERENCES neighborhoods(id),
  category          TEXT NOT NULL CHECK (category IN (
                      'most_trades',
                      'best_deals',
                      'best_stories',
                      'top_provisioner',
                      'longest_climb',
                      'highest_rated',
                      'most_generous',
                      'newest_legend'
                    )),
  period            TEXT NOT NULL CHECK (period IN
        ('weekly','monthly','alltime')),
  user_id           UUID NOT NULL REFERENCES users(id),
  rank              INTEGER NOT NULL,
  score             DECIMAL(10,2),
  snapshot_at       TIMESTAMPTZ DEFAULT now()
);
```

```sql
CREATE INDEX ON leaderboard_snapshots (neighborhood_id, category, period,
        snapshot_at DESC);
```

---

`invite_codes`

```sql
CREATE TABLE invite_codes (
  id                      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  code                    TEXT NOT NULL UNIQUE,
  issued_by_user_id       UUID NOT NULL REFERENCES users(id),
  neighborhood_id         UUID REFERENCES neighborhoods(id),
  used_by_user_id         UUID REFERENCES users(id),
  is_charter_member_invite BOOLEAN DEFAULT false,
  issued_at               TIMESTAMPTZ DEFAULT now(),
  used_at                 TIMESTAMPTZ,
  expires_at              TIMESTAMPTZ DEFAULT (now() + INTERVAL '30 days')
);
```

---

`feature_flags`

```sql
-- Server-side economy levers — adjustable without app update
CREATE TABLE feature_flags (
  key         TEXT PRIMARY KEY,
  value       TEXT NOT NULL,
  description TEXT,
  updated_at  TIMESTAMPTZ DEFAULT now()
);

INSERT INTO feature_flags VALUES
  ('eval_credit_reward',        '1',
        'Credits per evaluation (Tinder for Blenders)'),
  ('eval_comment_reward',       '5',   'Credits for evaluation with
        comment'),
  ('eval_daily_cap_free',       '25',  'Daily evaluation cap for free
        users'),
  ('eval_daily_cap_pro',        '40',  'Daily evaluation cap for pro
        users'),
  ('eval_daily_cap_legend',     '60',  'Daily evaluation cap for legend
        users'),
  ('signup_bonus',              '50',  'Credits on new user
        registration'),
  ('first_listing_bonus',       '10',  'Credits on first listing'),
  ('referral_bonus',            '50',  'Credits per referral (both
        parties)'),
  ('food_addon_reward_sender',  '5',   'Credits for sending a food add-
        on'),
```

```
  ('food_addon_reward_receiver','3',   'Credits for receiving a food add-
        on'),
  ('boost_cost',                '15',  'Credits to boost a listing for
        24hr'),
  ('relist_fee',                '5',   'Credits to relist an expired
        listing'),
  ('inactive_decay_days',       '90',  'Days of inactivity before decay
        begins'),
  ('inactive_decay_pct',        '10',  'Percentage of balance decayed per
        month');
```

---

### 3.2 Key Relationships Diagram

```
neighborhoods ←── users ──────────────┐
                     │                 │
                     ├── listings      │
                     │      │          │
                     │      ├── trades ┤
                     │      │     │    │
                     │      │     ├── messages
                     │      │     ├── ratings
                     │      │     └── credits_ledger entries
                     │      │
                     │      ├── evaluations
                     │      ├── listing_comments
                     │      ├── pitches
                     │      └── item_stories
                     │
                     ├── patches
                     ├── insider_accounts
                     ├── climb_chains ── climb_steps
                     ├── leaderboard_snapshots
                     └── invite_codes
```

---

## 4. API Design

### 4.1 Conventions

- **Transport:** HTTPS REST to Supabase Edge Functions for business logic. Direct Supabase JS client calls for simple CRUD.
- **Auth:** JWT in Authorization header (managed by Supabase Auth + Expo SecureStore).
- **Versioning:** `/v1/` prefix on all Edge Function endpoints.
- **Errors:** Standard JSON error envelope `{ error: { code, message, details } }`.

- **Pagination:** Cursor-based (not offset) using `created_at` as cursor for all feed queries.

### 4.2 Edge Function Endpoints

#### AI Listing Pipeline

```
POST /v1/ai-listing
Authorization: Bearer {jwt}
Content-Type: application/json

{
  "photo_urls": ["https://storage.supabase.co/..."],  // 1-3 uploaded
photos
  "neighborhood_id": "uuid"
}

Response 200:
{
  "title": "DeWalt 20V MAX Drill — Like New",
  "description": "...",
  "category": "tools",
  "condition": "like_new",
  "ai_suggested_price": 85,
  "ai_confidence": 0.87,
  "ebay_recent_sales": [90, 75, 95, 80],
  "ebay_avg": 85
}
```

#### Create Listing

```
POST /v1/listing
Authorization: Bearer {jwt}

{
  "title": "...",
  "description": "...",
  "category": "tools",
  "condition": "good",
  "credit_price": 85,
  "photos": ["url1", "url2"],
  "ai_suggested_price": 85,       // if user accepted AI price
  "is_perishable": false,
  "pitch_mode": false
}

Response 201:
```

```
{
  "listing_id": "uuid",
  "ai_seal": true,                 // true if price within 20% of AI
suggestion
  "credits_awarded": 10            // first listing bonus if applicable
}
```

**Request Trade**

```
POST /v1/trade/request
Authorization: Bearer {jwt}

{
  "listing_id": "uuid",
  "offer_type": "credit",
  "credit_amount": 85
}

Response 201:
{
  "trade_id": "uuid",
  "escrow_held": 85,
  "new_balance": 215
}

Error 402:
{
  "error": {
    "code": "INSUFFICIENT_CREDITS",
    "message": "You need 85 credits. You have 60.",
    "details": { "required": 85, "available": 60 }
  }
}
```

**Confirm Trade**

```
POST /v1/trade/{trade_id}/confirm
Authorization: Bearer {jwt}

{}  // no body needed; server knows who you are from JWT

Response 200:
{
  "both_confirmed": true,
  "credits_transferred": 85,
```

```
  "new_balance": 385,             // seller's new balance
  "patches_earned": ["first_fire"],
  "rank_changed": false
}
```

**Get Feed (The Market)**

```
GET /v1/feed?neighborhood_id={uuid}&radius_miles=1&cursor={created_at}
&limit=20&sort=newest
Authorization: Bearer {jwt}

Response 200:
{
  "listings": [...],
  "next_cursor": "2026-04-10T14:32:00Z",
  "has_more": true
}

// Sort options: newest | ending_soon | highest_value | ai_sealed |
boosted_first
```

**Get Credit Balance**

```
GET /v1/wallet/balance
Authorization: Bearer {jwt}

Response 200:
{
  "balance": 215,
  "escrow_held": 85,
  "spendable": 130,
  "ytd_trade_value_cents": 12400   // for 1099-B tracking display
}
```

### 4.3 Supabase Realtime Subscriptions (Client-side)

The mobile app subscribes to these channels using the Supabase Realtime client:

| Channel | Events | Used For |
|---|---|---|
| `trade:{trade_id}` | INSERT/UPDATE | Trade status changes, messages |
| `listing:{listing_id}` | UPDATE | Price changes, status updates, new comments |

| Channel | Events | Used For |
|---|---|---|
| `user:{user_id}:notifications` | INSERT | Push-equivalent in-app alerts |
| `neighborhood:{id}:listings` | INSERT | New listings in feed (optional, for feed refresh) |

---

## 5. AI Pipeline

### 5.1 Photo-to-Listing Pipeline

```
User taps "+" and takes 1-3 photos
          │
          ▼
[1] Upload to Supabase Storage
    • photos/{user_id}/{uuid}.jpg
    • Returns public URL(s)
          │
          ▼
[2] Call /v1/ai-listing Edge Function
          │
          ├──► [2a] Google Cloud Vision SafeSearch
          │         • Check for adult / violent content
          │         • If flagged: reject with "Nice try."
          │         • <200ms, free (1K/mo)
          │
          ├──► [2b] GPT-4o Vision (primary identification)
          │         Prompt:
          │         """
          │         Analyze this image. Return JSON:
          │         {
          │           "item_name": str,
          │           "brand": str | null,
          │           "model": str | null,
          │           "condition_estimate": "like_new|good|fair|
for_parts",
          │           "confidence": 0.0-1.0,
          │           "category": "tools|electronics|outdoor_camping|...",
          │           "notable_features": [str],
          │           "visible_damage": str | null
          │         }
          │         """
          │         Cost: ~$0.01-0.03 per call
```

```
          │
          ├──► [2c] eBay Browse API
          │         • Search: "{brand} {model} {item_name} used sold"
          │         • Filter: sold in last 30 days, shipped in US
          │         • Extract: last 5-10 sold prices
          │         • Free API
          │
          ├──► [2d] GPT-4o-mini (listing generation)
          │         Input: vision output + eBay price data
          │         Prompt:
          │         """
          │         Write a listing for The Manconomy, a neighborhood
          │         barter marketplace. Voice: direct, slightly humorous,
          │         honest about condition. Return JSON:
          │         {
          │           "title": str (max 60 chars),
          │           "description": str (2-4 sentences),
          │           "suggested_credit_price": int,
          │           "price_rationale": str (1 sentence)
          │         }
          │         Context: eBay recent sales avg $X, condition: Y
          │         """
          │         Cost: ~$0.001 per call (GPT-4o-mini is cheap)
          │
          └──► [2e] AI Seal evaluation
                    • If user-set price is within ±20% of
suggested_credit_price
                    • AND ai_confidence >= 0.70
                    • THEN ai_seal = true
          │
          ▼
[3] Return to client in <4 seconds
    {
      title, description, category, condition,
      ai_suggested_price, ai_confidence,
      ebay_avg, ebay_sales[]
    }
          │
          ▼
[4] User reviews, edits if desired, taps "List It"
          │
          ▼
[5] /v1/listing creates the record
    • ai_seal set based on final price vs suggestion
    • OpenAI Moderation API run on description text
    • If moderation flags: hold for review, don't publish
```

### 5.2 AI Seal Logic

```
function shouldAwardAiSeal(
  userPrice: number,
  aiSuggestedPrice: number,
  aiConfidence: number
): boolean {
  if (aiConfidence < 0.70) return false;
  const deviation = Math.abs(userPrice - aiSuggestedPrice) /
        aiSuggestedPrice;
  return deviation <= 0.20;  // within 20%
}
```

### 5.3 Crowd Valuation (Tinder for Blenders)

After 5+ independent evaluations on a listing, the `crowd_valuation` field is updated:

```
// Weighted verdict mapping
const VERDICT_MULTIPLIERS = {
  steal:    0.70,  // crowd thinks it's worth 30% more than listed
  fair:     1.00,
  too_high: 1.25,  // crowd thinks it should be 25% cheaper
  too_low:  0.85,
  skip:     null   // excluded from calculation
};

function computeCrowdValuation(
  listedPrice: number,
  verdicts: Verdict[]
): number {
  const valid = verdicts.filter(v => v !== 'skip');
  if (valid.length < 5) return listedPrice;  // not enough data

  const avg = valid.reduce((sum, v) =>
    sum + listedPrice * VERDICT_MULTIPLIERS[v], 0
  ) / valid.length;

  return Math.round(avg);
}
```

### 5.4 Fallback Behavior

| Failure | Behavior |
|---|---|
| GPT-4o Vision fails | |

| Failure | Behavior |
|---|---|
| | Show “We couldn’t identify this. Tell us what it is.” — manual entry mode |
| eBay API returns no results | Skip price validation; suggest credit price based on category baseline |
| AI confidence < 0.50 | Show “We’re not sure what this is — double check the details” — no AI Seal awarded |
| Moderation flags listing | Auto-hide, queue for manual review, notify user “Your listing is under review” |
| Full pipeline > 6 seconds | Show “The AI is taking a minute…” spinner; increase timeout to 15 seconds before fallback |

---

## 6. Credit Ledger Architecture

### 6.1 Design Principles

The credit ledger is the most critical system in The Manconomy. It must be:

- **Double-entry:** Every transaction has a source ( `from_user` ) and a destination ( `to_user` ). The system’s total credit supply is always knowable.
- **Immutable:** No records are ever updated or deleted. Status changes are new records.
- **Server-authoritative:** Only Edge Functions write to `credits_ledger`. The client never calls an INSERT directly.
- **Idempotent:** Every transaction has an `idempotency_key`. Retrying a failed request never double-spends.

### 6.2 Credit Transfer Flow (Trade Completion)

```
TRADE ACCEPTED (buyer puts credits in escrow):
  INSERT credits_ledger:
    from_user = buyer_id
    to_user   = 'escrow' (system UUID)
    amount    = 85
    type      = 'escrow_hold'
    reference_id = trade_id
    idempotency_key = uuid()

  buyer.spendable_balance decreases by 85

BOTH PARTIES CONFIRM:
```

```
  INSERT credits_ledger:
    from_user = 'escrow'
    to_user   = seller_id
    amount    = 85
    type      = 'escrow_release'
    reference_id = trade_id
    idempotency_key = uuid()

  escrow balance = 0; seller balance increases by 85

TRADE CANCELLED:
  INSERT credits_ledger:
    from_user = 'escrow'
    to_user   = buyer_id
    amount    = 85
    type      = 'escrow_return'
    reference_id = trade_id
    idempotency_key = uuid()

  escrow balance = 0; buyer gets credits back
```

### 6.3 Credit Balance Calculation

```
-- Spendable balance for a user
-- (total received) - (total sent) - (credits in active escrow)
SELECT
  COALESCE(SUM(CASE WHEN to_user = $1 THEN amount ELSE 0 END), 0) -
  COALESCE(SUM(CASE WHEN from_user = $1 THEN amount ELSE 0 END), 0) -
  (
    SELECT COALESCE(SUM(amount), 0)
    FROM credits_ledger cl
    JOIN trades t ON t.id = cl.reference_id
    WHERE cl.from_user = $1
      AND cl.type = 'escrow_hold'
      AND t.status IN ('requested', 'accepted')
  ) AS spendable_balance
FROM credits_ledger
WHERE to_user = $1 OR from_user = $1;
```

### 6.4 Anti-Fraud Checks (Server-side, before any credit mutation)

```
async function validateCreditTransfer(
  buyerId: string,
  amount: number,
```

```
  tradeId: string,
  idempotencyKey: string
): Promise<{ valid: boolean; error?: string }> {

  // 1. Check idempotency — has this key been used before?
  const existing = await db.query(
    'SELECT id FROM credits_ledger WHERE idempotency_key = $1',
    [idempotencyKey]
  );
  if (existing.rows.length > 0) {
    return { valid:
        true };  // idempotent repeat — return success without re-
        processing
  }

  // 2. Check buyer has sufficient spendable balance
  const balance = await getSpendableBalance(buyerId);
  if (balance < amount) {
    return { valid: false, error: 'INSUFFICIENT_CREDITS' };
  }

  // 3. Check this trade hasn't already been transferred
  const existingTransfer = await db.query(
    `SELECT id FROM credits_ledger
     WHERE reference_id = $1 AND type = 'escrow_hold'`,
    [tradeId]
  );
  if (existingTransfer.rows.length > 0) {
    return { valid: false, error: 'TRADE_ALREADY_ESCROWED' };
  }

  // 4. Check for wash trade pattern
  const recentMutualTrades = await db.query(
    `SELECT COUNT(*) FROM trades
     WHERE ((buyer_id = $1 AND seller_id = $2)
        OR (buyer_id = $2 AND seller_id = $1))
       AND status = 'completed'
       AND completed_at > now() - INTERVAL '30 days'`,
    [buyerId, /* sellerId from trade record */]
  );
  if (parseInt(recentMutualTrades.rows[0].count) >= 3) {
    // Flag for manual review, but don't block
    await flagForReview(tradeId, 'WASH_TRADE_PATTERN');
  }

  return { valid: true };
}
```

## 7. Security Model

### 7.1 Row Level Security (RLS) Policies

All tables have RLS enabled. Key policies:

```sql
-- Users: can only update their own row
ALTER TABLE users ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users can read all public profiles"
  ON users FOR SELECT TO authenticated
  USING (true);

CREATE POLICY "Users can update own profile"
  ON users FOR UPDATE TO authenticated
  USING (auth.uid() = id);

-- Listings: readable by same neighborhood users
ALTER TABLE listings ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users can read listings in their neighborhood"
  ON listings FOR SELECT TO authenticated
  USING (
    neighborhood_id = (SELECT neighborhood_id FROM users WHERE id =
        auth.uid())
    OR user_id = auth.uid()
  );

CREATE POLICY "Users can create their own listings"
  ON listings FOR INSERT TO authenticated
  WITH CHECK (user_id = auth.uid());

CREATE POLICY "Users can update own listings"
  ON listings FOR UPDATE TO authenticated
  USING (user_id = auth.uid());

-- Trades: only buyer and seller can read
ALTER TABLE trades ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Trade parties can read trades"
  ON trades FOR SELECT TO authenticated
  USING (buyer_id = auth.uid() OR seller_id = auth.uid());

-- Credits ledger: users can only read their own ledger entries
```

```sql
ALTER TABLE credits_ledger ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users read own ledger entries"
  ON credits_ledger FOR SELECT TO authenticated
  USING (from_user = auth.uid() OR to_user = auth.uid());

-- NO INSERT policy for authenticated users on credits_ledger
-- All writes go through SECURITY DEFINER Edge Functions only

-- Messages: only trade participants can read
ALTER TABLE messages ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Trade participants read messages"
  ON messages FOR SELECT TO authenticated
  USING (
    EXISTS (
      SELECT 1 FROM trades
      WHERE trades.id = messages.trade_id
        AND (trades.buyer_id = auth.uid() OR trades.seller_id =
        auth.uid())
    )
  );

CREATE POLICY "Trade participants send messages"
  ON messages FOR INSERT TO authenticated
  WITH CHECK (
    sender_id = auth.uid()
    AND EXISTS (
      SELECT 1 FROM trades
      WHERE trades.id = messages.trade_id
        AND (trades.buyer_id = auth.uid() OR trades.seller_id =
        auth.uid())
    )
  );
```

### 7.2 Authentication Flow

```
1. User opens app → check SecureStore for existing session
2. If no session → login/register screen
3. Email/password or Google/Apple SSO → Supabase Auth
4. Phone verification required before listing or trading:
   - Send SMS code via Supabase (Twilio integration)
   - Code must be verified within 10 minutes
   - Sets users.is_verified = true
5. JWT stored in Expo SecureStore (never in AsyncStorage)
6. JWT refreshes automatically via Supabase client SDK
```

```
7. On API calls: JWT sent as Authorization: Bearer {token}
8. Supabase verifies JWT signature; RLS uses auth.uid()
```

### 7.3 Secrets Management

| Secret | Storage | Access |
|---|---|---|
| OpenAI API key | Supabase Edge Function env vars | Edge Functions only, never client |
| eBay API key | Supabase Edge Function env vars | Edge Functions only |
| Google Cloud Vision API key | Supabase Edge Function env vars | Edge Functions only |
| Stripe secret key | Supabase Edge Function env vars | Edge Functions only |
| Stripe webhook secret | Supabase Edge Function env vars | Webhook handler only |
| Supabase service role key | Server only (never in client bundle) | Edge Functions only |
| Supabase anon key | Client bundle (safe — RLS enforced) | Mobile app |

### 7.4 Anti-Gaming & Fraud Controls

| Attack Vector | Detection | Response |
|---|---|---|
| Mass-swipe rating spam | Average swipe time < 1.5s over 10 consecutive evals | CAPTCHA + warning; repeat → 24hr eval suspension |
| Multi-account signup bonus | One phone number per account; device fingerprint check | Block second account; flag for review |
| Wash trades | Same-pair trade count >3 in 30 days | Flag for manual review; no automatic block |
| Self-trading | `buyer_id != seller_id` check in DB constraint | Blocked at schema level |
| Same-address trading | Address proximity check on user location | Flagged; not blocked (neighbors legitimately trade) |
| Insider credit farming | Insider can only review linked user's listings; 10/day cap | Enforced in Edge Function, not client |
| Referral abuse | | |

| Attack Vector | Detection | Response |
|---|---|---|
| | Referee must complete trade with someone OTHER than referrer | Checked in `award-referral-bonus` Edge Function |

---

## 8. Infrastructure

### 8.1 Stack Summary

| Layer | Technology | Tier |
|---|---|---|
| Mobile | React Native 0.74 + Expo SDK 51 | — |
| Navigation | Expo Router 3.x | — |
| State | Zustand 4.x | — |
| Maps | react-native-maps 1.x | — |
| Backend | Supabase (managed cloud) | Pro ($25/mo) |
| Database | PostgreSQL 15 + PostGIS | Included in Supabase |
| Realtime | Supabase Realtime | Included in Supabase |
| Auth | Supabase Auth | Included in Supabase |
| File storage | Supabase Storage (or Cloudflare R2) | Included / $0 egress |
| Edge functions | Deno (via Supabase) | Included |
| AI — vision | OpenAI GPT-4o Vision | Pay-per-use ~$0.02/call |
| AI — text | OpenAI GPT-4o-mini | ~$0.001/call |
| AI — moderation | OpenAI Moderation API | Free |
| Image moderation | Google Cloud Vision SafeSearch | 1K/mo free |
| Price validation | eBay Browse API | Free |
| Payments | Stripe | 2.9% + $0.30/transaction |
| Push notifications | Expo Push Notifications | Free |
| Analytics | PostHog | Free tier (1M events/mo) |
| Error tracking | Sentry | Free tier (5K events/mo) |

| Layer | Technology | Tier |
|---|---|---|
| CI/CD | GitHub Actions + Expo EAS | Free / $15/mo EAS Pro |

### 8.2 Environments

| Environment | Purpose | Supabase Project |
|---|---|---|
| `development` | Local dev; developer's machine | Local Supabase CLI |
| `staging` | Pre-release testing; seeded test data | Separate Supabase project (free tier) |
| `production` | Live app | Supabase Pro project |

### 8.3 Monthly Cost Projection

| Scale | Supabase | OpenAI | Storage | Other | Total |
|---|---|---|---|---|---|
| 100 active users | $25 | ~$5 | ~$2 | ~$8 | **~$40/mo** |
| 500 active users | $25 | ~$25 | ~$8 | ~$12 | **~$70/mo** |
| 1,000 active users | $25 | ~$50 | ~$15 | ~$20 | **~$110/mo** |
| 5,000 active users | $25 | ~$200 | ~$40 | ~$35 | **~$300/mo** |

> *Break-even occurs at ~350–500 active users at conservative revenue projections (~$1,490 MRR per 1K users).*

### 8.4 CI/CD Pipeline

```
Developer pushes to feature branch
  |
  ▼
GitHub Actions: lint + type-check (tsc --noEmit)
  |
  ▼  (on merge to main)
GitHub Actions: run test suite
  |
  ▼
Expo EAS Build: create preview build
  |
  ▼  (on tag: v*.*.*)
Expo EAS Build: production build
  |
```

```
    ├── iOS: submit to App Store via EAS Submit
    └── Android: submit to Play Store via EAS Submit
```

### 8.5 Database Migrations

Use Supabase CLI for migrations:

```
supabase migration new create_initial_schema
supabase db push                              # apply to staging
supabase db push --db-url $PROD_DB_URL       # apply to production
```

All schema changes go through migration files — no direct DDL in production.

---

## 9. Key Technical Decisions

*(See `docs/adr/` for full ADR documents)*

| Decision | Choice | Short Rationale |
|---|---|---|
| Backend platform | Supabase | Postgres + Auth + Realtime + Storage in one; open-source; free tier; no vendor lock-in |
| Mobile framework | React Native + Expo | Single iOS/Android codebase; Expo simplifies build/OTA; largest community; JS/TS = one language |
| Credit architecture | Closed-loop, non-convertible | Avoids money transmitter licensing; same legal model as airline miles; Stripe handles all cash |
| Primary database | PostgreSQL | Relational model fits trade/credit relationships; PostGIS for geospatial; strong ecosystem |
| API style | REST over GraphQL | Simpler to debug for solo dev; Supabase client handles most queries; GraphQL overhead not worth it at MVP |
| AI provider | OpenAI (GPT-4o) | Best vision model available; reliable API; $0.01-0.03/call is economically viable at scale |

| Decision | Choice | Short Rationale |
|---|---|---|
| File storage | Supabase Storage (or Cloudflare R2) | Zero egress on R2; Supabase Storage simpler for MVP; can migrate to R2 later |
| State management | Zustand | Minimal boilerplate vs Redux; simple enough for this use case |
| Payments | Stripe | Platform never holds money; Stripe handles PCI compliance, 1099-K, Apple Pay, Google Pay |

---

## 10. Open Questions

These require decisions before or during build:

### Architecture / Data

| # | Question | Impact | Decision Needed By |
|---|---|---|---|
| OQ-1 | Should listing photos go to Supabase Storage or Cloudflare R2? Supabase Storage is simpler to set up; R2 has zero egress fees and will be cheaper at scale. | Cost at scale | Before Sprint 3 |
| OQ-2 | Should `escrow` be a system UUID in `credits_ledger` or a separate `escrow_accounts` table? A dedicated table is more explicit; system UUID is simpler. | Credit ledger integrity | Before Sprint 1 |
| OQ-3 | How do we handle the 48-hour auto-confirm cron at MVP scale? Supabase doesn't have a native cron scheduler in the | Trade flow reliability | Before Sprint 6 |

| # | Question | Impact | Decision Needed By |
|---|---|---|---|
| | free tier; pg_cron is available in Pro tier. | | |
| OQ-4 | Do leaderboards run as real-time views or nightly snapshot jobs? Real-time is more accurate but expensive; snapshots are cheaper and “good enough” for weekly boards. | Performance | Before Sprint 8 (leaderboards) |

### Product / Legal

| # | Question | Impact | Decision Needed By |
|---|---|---|---|
| OQ-5 | At exactly what FMV threshold do we collect tax ID from users? $500 as a warning, $600 as a block. Confirm with legal. | 1099-B compliance | Before launch |
| OQ-6 | Is the homebrew alcohol disclaimer sufficient in all 50 states, or do we need state-specific variations? | Legal liability | Before food listing launch |
| OQ-7 | Should invite codes be scoped to a neighborhood or global? Neighborhood-scoped creates tighter communities; global is simpler. | Community architecture | Before Sprint 1 |
| OQ-8 | What is the minimum neighborhood size to open enrollment? 200 active members per plan. Is this too high for the first launch? | Cold start | Before launch |

| # | Question | Impact | Decision Needed By |
|---|---|---|---|
| | Consider 100 as the initial threshold. | | |

### UX / App Store

| # | Question | Impact | Decision Needed By |
|---|---|---|---|
| OQ-9 | Age rating: 17+ (for marketplace with user-generated content + alcohol category) or 12+? | App Store submission | Before App Store submission |
| OQ-10 | Do we allow in-app photo editing (crop/brightness) or just raw camera photos? Editing improves listing quality; adds build time. | Listing quality | Before Sprint 3 |

---

*Document version 1.0 — April 11, 2026*
*See also:* `docs/product/mvp-spec.md`, `docs/product/master-plan.md`, `docs/adr/`